Dokumentasi Tool: mobile_audit.py

Nama tool: mobile_audit.py (otomasi reverse engineering + secret scanning)

Lokasi: `tools/mobile/mobile_audit.py` (wrapper: `tools/mobile_audit.py`)

Bahasa: Python 3 (murni, minim dependency opsional: lief / cryptography / androguard)

Slash command: `/mobile-audit <apk|ipa|aab|dll|exe|so|hbc|jar|dir>`

Target: APK / AAB / XAPK / APKS / IPA / JAR / DEX / SMALI / SO (ELF) / DLL / EXE (PE) / Mach-O / HBC (Hermes/React Native) / direktori hasil ekstraksi

---

1. Ringkasan

`mobile_audit.py` adalah alat otomasi **reverse engineering** untuk aplikasi mobile dan biner

yang kami buat sendiri. Sekali jalan, tool ini:

1. Mengenali jenis arsip/biner & framework aplikasi (Flutter, React Native, Unity, Xamarin, Cordova, .NET, native).

2. Membongkar & mendecompile per format ke *corpus* teks yang bisa dibaca.

3. Men-scan seluruh corpus terhadap **database regex hardcoded-secrets** (91 rule / 38 kategori).

4. Membuat laporan **Markdown + JSON** yang terstruktur.

Alur kerja tool mengikuti 4 fase: **IDENTIFY -> UNPACK -> SCAN -> REPORT**.

Tool ini dipakai nyata untuk mengaudit aplikasi Sapa Pegadaian (hasilnya di

`reports/sapa_re/`) - contoh eksekusi nyata dari tool ke aplikasi Android produksi.

---

2. Struktur Kode

```
tools/
■■■ mobile_audit.py              # wrapper CLI -> tools/mobile/mobile_audit.py
■■■ mobile/
    ■■■ mobile_audit.py           # orchestrator utama (IDENTIFY -> UNPACK -> SCAN -> REPORT)
    ■■■ secret_scanner.py         # engine scan regex (Rule, Finding, scan_corpus, summarize)
    ■■■ rules/
    ■   ■■■ secrets.json          # database regex 91 rule (edit bebas untuk tambah rule)
    ■■■ engines/                  # engine per-format
    ■   ■■■ __init__.py
    ■   ■■■ common.py             # find_tool(), run(), extract_strings_basic(), log()
    ■   ■■■ fingerprint.py        # magic sniff + deteksi framework + rabin2 identity
    ■   ■■■ apk_engine.py         # bongkar APK/AAB, dispatch artefak, apktool decode, signing
    ■   ■■■ ipa_engine.py         # bongkar IPA (Payload/*.app, Mach-O, dylib, plist)
    ■   ■■■ dex_engine.py         # DEX -> Java (jadx), fallback Smali (apktool)
    ■   ■■■ native_engine.py      # ELF/Mach-O/PE -> strings (rabin2 -z) + symbols (lief)
    ■   ■■■ dotnet_engine.py      # .NET -> C# (ilspycmd)
    ■   ■■■ hermes_engine.py      # .hbc -> Hermes asm (hbctool), fallback strings
    ■   ■■■ flutter_engine.py     # libapp.so -> blutter (Dart symbols), fallback strings
    ■   ■■■ unity_engine.py       # libil2cpp.so + global-metadata.dat -> dump.cs (Il2CppDumper)
    ■   ■■■ generic_engine.py     # biner tak dikenal -> dump strings manual
    ■   ■■■ plist_engine.py       # plist biner -> XML (plistlib)
    ■■■ reports/                  # output default per-target
    ■■■ bin/                      # tool terpasang offline (lihat Bab 6)
```

---

3. Cara Kerja Tool (Flow Reverse Engineering)

3.1 Phase 1 - IDENTIFY

Orchestrator memanggil `classify(target)`:

- Jika target **direktori** -> `detect_framework()` (deteksi framework dari isi folder).
- Jika target **file** -> `sniff_file()` baca magic bytes 8 byte pertama, dicocokkan ke tabel

`MAGIC` di `engines/fingerprint.py`:

Magic	Jenis
---	---
7F 45 4C 46	ELF (.so native)
FE ED FA CF/CE (BE) / CF FA ED FE/CE (LE)	Mach-O (iOS)
4D 5A (MZ)	PE (.dll / .exe)
50 4B 03 04 (PK)	ZIP / APK / JAR
64 65 78 0A (dex\n)	DEX
1F BC 0C 00	Hermes bytecode (.hbc)
CA FE BA BE	.class
02 00 0C 00	ARSC (resources)
03 00 08 00 / 01 03 00 08	AXML (binary XML)
1F 8B	GZIP

Klasifikasi akhir `classify()` memadukan **ekstensi** + **magic** (mis. `.apk` -> apk, `.ipa` -> ipa, `.so`/ELF -> elf, `.dll`/`.exe` + PE -> pe, dst). Untuk PE, `_is_dotnet()` mendeteksi .NET via penanda `BSJB` (header CLI metadata) atau `mscoree`/`_CorDllMain`.

Untuk direktori hasil ekstraksi, `detect_framework()` mencari penanda framework:

Penanda file	Framework terdeteksi
---	---
libapp.so + libflutter.so	Flutter (Dart compiled)
libil2cpp.so (+ global-metadata.dat)	Unity IL2CPP
assembly-csharp.dll	Unity Mono
index.android.bundle (+ magic HBC)	React Native (Hermes) / plain JS
*.hbc	React Native (Hermes bytecode)
assets/www/index.html	Cordova / Ionic / Capacitor
libmonodroid.so / libmono.so	Xamarin (.NET on Android)
.dll / .exe	.NET managed assembly
tidak ada penanda	Native binary / unknown

3.2 Phase 2 - UNPACK

Bergantung tipe, tool membongkar dan mendecompile per format:

- **APK / AAB / XAPK / APKS** (`apk_engine.unpack_apk`):

1. Ekstrak zip -> `apk_root`.

2. `*.dex` -> `dex_engine`: **jadx** (`--no-res --no-debug-info`) -> `corpus/sources_java` (Java). Bila jadx gagal -> fallback **apktool** `-s` -> Smali.

3. `AndroidManifest.xml` -> **apktool decode** -> `corpus/decoded_res` (XML terbaca).

4. `*.so` -> dispatch: `libapp.so` (Flutter) ke flutter_engine, `libil2cpp.so` (Unity) ke unity_engine, sisanya ke native_engine.

5. Aset teks (`.js .json .xml .txt .properties .yml .yaml .ini .html .css .conf .cfg

.env .plist .ts .php .py .rb .md) -> disalin ke corpus/assets/`.

6. Biner tak dikenal (bukan teks, bukan .so/.bin/.dat/.ttf/.png/.jpg) -> generic_engine (dump strings) ke `corpus/strings_other/`.

7. `META-INF/*.RSA` -> catat info signing.

- **IPA** (`ipa_engine.unpack_ipa`):

1. Ekstrak zip -> `ipa_root`, cari `Payload/*.app`.

2. Biner Mach-O utama (file tanpa ekstensi di root `.app`) + semua `.dylib`/framework biner -> native_engine.

3. Semua `.plist` + `.mobileprovision` -> plist_engine (plist biner -> XML).

4. Aset teks -> `corpus/assets/`.

- **JAR**: ekstrak zip + jadx `--no-res` -> Java source.
- **DEX**: langsung jadx -> Java (fallback smali).
- **ELF / PE / Mach-O** (`native_engine.run`):
- strings via **rabin2 -z -qq** -> `corpus/strings_native/*.strings`

(fallback: ekstraksi strings manual `extract_strings_basic`, min 6 char).

- symbols & imports via **lief** -> `corpus/symbols_native/*.symbols`.
- **PE .NET** (`dotnet_engine.run`): **ilspycmd -p** -> C# source ke `corpus/sources_dotnet/` (fallback dump single-file).
- **HBC** (`hermes_engine.run`): **hbctool** disasm -> `corpus/hermes_asm/*.hbcasm` (fallback strings).
- **AXML** (`_dump_axml`): **androguard** AXMLPrinter -> XML.
- **Direktori**: `apk_engine.dispatch_extracted` langsung (re-scan artefak).
- **Tak dikenal**: generic_engine (strings manual, max 30 MB).

Seluruh output korpus ini yang menjadi bahan scan (workdir dihapus otomatis setelah selesai, kecuali `--keep`).

3.3 Phase 3 - SCAN (`secret_scanner.py`)

- Memuat rules dari `rules/secrets.json` (91 rule aktif).
- `iter_scan_files()`: lewati ekstensi biner (`.png .jpg .so .dll .dex .jar` dll), lewati file > 15 MB, lewati folder `.git`.
- Dekode tiap file dengan **utf-8 lalu latin-1**; hitung jumlah file & baris (stats).
- Untuk setiap rule: jalankan `regex.finditer` (flag DOTALL), cap **30 match per file**, petakan match ke nomor baris + konteks baris.
- **Filter noise library**: rule dengan flag `skip_libraries: true` otomatis dilewati untuk path pustaka/3rd-party (`/meta-inf/`, `/third_party_licenses/`, `/sources/androidx/`, `/sources/com/google/`, okhttp3, retrofit2, io/realm, kotlin, dll - daftar di `LIBRARY_NOISE_PATHS`) + `skip_paths` tambahan per-rule.

3.4 Phase 4 - REPORT

`write_report()` menghasilkan dua file di folder output:

- `report.md` - laporan terbaca manusia:
- Header: target, tipe, framework, deteksi, jumlah file/baris.
- Tabel ringkasan **severity** (Critical/High/Medium/Low/Info) & **kategori**.
- Tabel **tooling terpasang** (status masing-masing tool di `bin/`).
- **Detail per rule**: deskripsi rule, total lokasi, dan 20 contoh lokasi

`file:baris :: match` (capped), plus "... dan N lagi".

- `findings.json` - data terstruktur untuk automation:

`target`, `type`, `frameworks`, `stats`, `summary` (severity & kategori), `findings[]`

(rule, category, severity, description, file, line, match, context).

Sorting: severity critical -> high -> medium -> low -> info.

---

4. Database Rules (rules/secrets.json)

- **91 rule aktif**, komposisi severity: critical 37, high 28, medium 14, low 12.
- **38 kategori**: AWS, GCP, Firebase, Azure, GitHub, Slack, Stripe, Twilio, SendGrid,

OpenAI, Anthropic, Telegram, Discord, JWT, Auth, Crypto, Credentials, Database, Email,

Cloudflare, DigitalOcean, npm, PyPI, Shopify, Square, PayPal, Facebook, Apple, Alibaba,

Heroku, SonarQube, Jenkins, Grafana, Sentry, MinIO, Android, Endpoints, PII, Identifiers.

- Contoh rule penting: `aws_access_key_id` (AKIA...), `jwt_bearer` (3 segmen base64url),

`rsa_private_key`/`ec_private_key`/`openssh_private_key`, `hardcoded_password`

(password=/pwd= wajib ada digit), `retrofit_api_path` (anotasi @GET/@POST dst,

skip_libraries: true).

Format rule (JSON)

```
{
  "id": "aws_access_key_id",
  "category": "AWS",
  "severity": "high",
  "description": "AWS Access Key ID (AKIA...)",
  "pattern": "\\bAKIA[0-9A-Z]{16}\\b",
  "enabled": true,
  "flags": 0,
  "skip_libraries": false,
  "skip_paths": []
}
```

Field: `id` (unik), `category`, `severity` (critical/high/medium/low/info), `description`,

`pattern` (regex Python), `enabled` (false = dinonaktifkan tanpa dihapus), `flags`

(regex flags, mis. 2 = IGNORECASE), `skip_libraries` (skip path noise pustaka),

`skip_paths` (path tambahan yang di-skip).

Cara menambah rule: edit `rules/secrets.json`, tambah objek dengan `enabled: true`,

jalankan ulang tool (atau lewat `--rules file_custom.json`).

---

5. Cara Pakai

5.1 Prasyarat

- Python 3.10+ (tools berbasis 3.14 dipakai di lingkungan ini).
- Tool RE di `tools/mobile/bin/` (sudah dibundle): jadx, apktool, radare2 (rabin2),

ghidra, ilspycmd + dotnet-sdk, Il2CppDumper. Bila tidak ada, tool tetap jalan dengan

fallback (strings manual / smali / warning).

- `java` di PATH untuk apktool.
- Opsional: `lief` (symbols), `cryptography` (signing), `androguard` (AXML), `hbctool`

(Hermes), `blutter` (Flutter - perlu Rust+MSVC).

5.2 Perintah

```
python tools/mobile_audit.py <target> [opsi]
```

Opsi	Fungsi
---	---
-o, --out DIR	Direktori output (default: tools/mobile/reports/<target>)
--rules FILE	Rules JSON custom (default: rules/secrets.json)
--no-unpack	Lewati unpack; scan corpus/direktori yang sudah ada
--quiet	Kurangi log
--keep	Pertahankan artefak unpack (work/) setelah selesai

5.3 Contoh

```
# APK Android -> laporan default
python tools/mobile_audit.py app.apk

# IPA iOS dengan output spesifik
python tools/mobile_audit.py app.ipa -o reports/ios_audit

# Biner native tunggal
python tools/mobile_audit.py lib/arm64-v8a/libnative.so

# .NET assembly
python tools/mobile_audit.py bin/MyApp.dll

# Hermes bundle React Native
python tools/mobile_audit.py assets/index.android.bundle

# Scan ulang corpus hasil ekstraksi (tanpa bongkar lagi)
python tools/mobile_audit.py extracted_app_dir --no-unpack

# Rules khusus
python tools/mobile_audit.py app.apk --rules my_rules.json --keep
```

5.4 Via slash command

```
/mobile-audit <apk|ipa|aab|dll|exe|so|hbc|jar|dir>
```

5.5 Contoh output nyata (uji: InjuredAndroid.apk)

- Tipe: `apk`, Framework: `Flutter (Dart compiled)`.
- **2568 file / 390.247 baris** discan.
- Ringkasan: High 8, Medium 4, Low 744 (677 di kategori Endpoints).
- Semua tooling terpasang terdeteksi (jadx, apktool, rabin2, ghidra, ilspycmd,

il2cppdumper); blutter belum ada.

---

6. Tool Terbundle (bin/)

Tool	Fungsi	Dipakai oleh
---	---	---
jadx	DEX/JAR -> Java source	dex_engine, _unpack_jar
apktool	Decode resource + Smali	apk_engine (manifest), dex_engine (fallback smali)
radare2 (rabin2)	strings + info biner ELF/Mach-O/PE	native_engine, fingerprint (identity)
ghidra (analyzeHeadless)	analisis deep biner (manual)	terdeteksi di tool_map
ilspycmd + dotnet-sdk	.NET -> C#	dotnet_engine
Il2CppDumper	Unity IL2CPP -> dump.cs	unity_engine

`find_tool()` di `engines/common.py` mencari tool dengan glob ke `bin/` lalu cache hasilnya (sehingga tidak perlu PATH system).

---

7. Kelebihan

1. **Multi-format satu perintah** - APK/AAB/IPA/JAR/DEX/ELF/PE/Mach-O/HBC/direktori ditangani otomatis tanpa konfigurasi.
2. **Framework-agnostic** - Flutter, React Native (Hermes), Unity (IL2CPP/Mono), Xamarin, Cordova, .NET, native: semua terdeteksi & di-dispatch ke engine yang tepat.
3. **Tool lengkap ter-bundle offline** di `bin/` - tidak perlu install jadx/apktool/dll; otomatis terdeteksi & di-cache.
4. **Fallback berlapis** - jadx->smali, rabin2->strings manual, blutter->strings, hbctool->strings; tool tidak mati saat satu tool hilang.
5. **Rules terpusat & mudah di-extend** - 91 rule JSON per kategori; tambah/ubah rule tanpa menyentuh kode; bisa pakai --rules custom.
6. **Filter noise library** (skip_libraries) - menekan false positive dari kode pustaka (okhttp, retrofit, androidx, kotlin, dll).
7. **Output ganda** - report.md untuk manusia + findings.json terstruktur untuk automation/CI.
8. **Tampilan nyaman** - laporan dikelompokkan per rule, diranking severity, dengan contoh lokasi; stats file/baris tercatat.
9. **Minim dependency** - Python murni; hanya opsional (lief/cryptography/androguard).
10. **Beres Windows-first** - CREATE_NO_WINDOW, pencari .bat/.exe, path dengan glob.

---

8. Kekurangan / Batasan

1. **Tanpa deobfuscation** - kode yang di-proguard/R8/minify atau string-terenkripsi banyak yang lolos (string secret disembunyikan runtime).
2. **Tanpa dynamic analysis** - frida/objection hanya terdaftar sebagai tool yang dicari, belum dipakai; secret yang hanya muncul saat runtime tidak terlihat.
3. **Konfirmasi validitas secret minim** - rule regex menemukan kandidat, tetapi tidak ada

verifikasi aktif (mis. cek ke API provider) apakah secret masih hidup/valid.

4. **FP pada rule longgar** - contoh nyata: 677 temuan kategori Endpoints pada InjuredAndroid; perlu triase manual.
5. **Pembatasan scan** - file > 15 MB di-skip; **max 30 match per rule per file** (data besar terpotong); konteks match di-cap 200/300 char.
6. **Sumber asli hilang** - biner hanya di-dump strings; tidak ada relokasi/offset ke disassembly (perlu ghidra manual untuk lanjutan).
7. **dedupe() ada tapi belum dipakai di pipeline** - laporan masih per-file, bukan deduplicated lintas run.
8. **Timeouts tetap** - jadx 900s / apktool 600s; APK raksasa bisa gagal sebagian (masih ada fallback).
9. **Ketergantungan java** untuk apktool; **blutter** butuh Rust+MSVC (belum ada di bin/).
10. **Tidak menangani** unpacking UPX, aplikasi encrypted/signed-exotic, atau patch checksum; output report berbahasa Indonesia (perlu penyesuaian untuk tim global).
11. **Belum ada mode batch / differential** antar versi APK untuk melihat perubahan.

---

9. Peta Perbaikan (Roadmap)

- Validasi secret aktif (call provider API / test token) untuk menekan FP.
- Integrasi **frida/objection** hook untuk dynamic secret extraction & bypass.
- **Deobfuscation string** (proguard dictionary, DexGuard) + resurreksi dari APK asli.
- Memakai `dedupe()` dan severity scoring otomatis di laporan.
- Mode **batch multi-target** dan **differential analysis** antar versi.
- Scan resources biner (ARSC) & offset mapping ke disassembly (rabin2 -S).
- Dukungan UPX unpack + penanganan APK termodifikasi (checksum).

Dokumen disusun dari pembacaan langsung implementasi tool (orchestrator, scanner, engine per-format, rules JSON, dan contoh report InjuredAndroid).